
building-agents

Build your own agent by building the harness around a model.

01 · WHO I AM

Chase Dovey

I research agentic systems.
Most of my work is building
harnesses around foundational
models.

02 · THE LAB

Average Joes Lab

A private citizen research lab.
Independent research on
agentic systems, published
openly.

03 · THIS TALK

Agent = Model + Harness

The 3 disciplines and the 10-
module curriculum that builds a
production-shaped harness
from a single LLM call.

github.com/averagejoeslab/building-agents

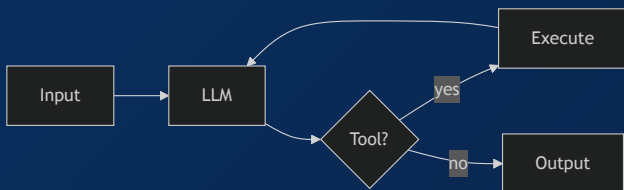
What are agentic systems?

Systems that act on their own. The agency comes from an LLM coordinating calls to reach a goal without supervision.

WORKFLOW - CODE DECIDES THE PATH



AGENT - MODEL DECIDES THE PATH



I focus on agents.

Systems with autonomy over their own control flow.

EXAMPLES IN THE WILD

- Claude Code
- Cursor
- Devin
- Aider
- openclaw

The three disciplines

Three layers stacked. You go through them in order – from nothing to agent-built software.

01 · OUTPUT: CALLABLE MODEL

Model development

A handful of labs train the foundational model.

GPT · Claude · Gemini · Llama

02 · OUTPUT: AN AGENT

Harness engineering

Wrap the model in code, state, tools, loop.

Agent = Model + Harness

Claude Code · Cursor · Codex

← **this talk's focus**

03 · OUTPUT: PRODUCTS BUILT BY AGENTS

Agentic engineering

Use the agent to build other software, products, agents.

openclaw · vibe coding's pro cousin

Discipline 1 • Model development

A handful of labs (Anthropic, OpenAI, Google, Meta) train foundational models. The output is a service you call by API.

ARCHITECTURE • HOW IT ANSWERS

PRE • NOT LEARNED

Input layer • your prompt arrives



Tokenizer • text → number IDs (BPE)



NEURAL NETWORK • LEARNED DURING TRAINING

Embedding • IDs → "meaning vectors"



Positional encoding • adds word-order info (RoPE)



Transformer block × 30-100

the "thinking" – every word looks at every other, refining meaning over and over



LM head • score every possible next word



POST • NOT LEARNED

Output layer • sample one word, return text → loop

TRAINING • HOW IT LEARNED (SHAPES THE NEURAL NETWORK)

1

Pretraining

Next-token prediction over trillions of web tokens – produces the base model.

2

Mid-training

Continued pretraining on curated data – sharpens code, math, reasoning.

3

Supervised Fine-Tuning (SFT)

Instruction/response pairs – teach the model to follow instructions.

4

Preference tuning • RLHF / DPO / GRPO

Human-rated comparisons – align outputs to be helpful, honest, harmless.

5

Constitutional AI / RLAI

AI feedback against written principles – scales alignment past human-only labeling.

6

Reasoning RL • GRPO + verifiable rewards

Math/code with rule-based rewards – trains explicit chain-of-thought (R1, o-series).

We don't teach this. The harness layer assumes it's already happened **upstream**.

Discipline 2 · Harness engineering

Agent = Model + Harness

The harness is every piece of code, configuration, and execution logic that isn't the model itself.

THE 9 PRIMARY COMPONENTS

Model interface

How the harness calls the LLM – which model, sync or streaming, response parsing.

Control flow

The loop that drives the model continuously – the TAO loop is the workhorse.

Memory + context

What persists across sessions; what fits into each call's token budget.

Tools · action layer

The model's hands – functions it can invoke to affect the world.

Execution environment

Where dangerous tool calls actually run – sandbox, container, isolated VM.

Safety · guardrails

What the model is allowed to do – approval gates, loop bounds, retry policy.

Observability

Structured traces of every LLM call, tool call, and state transition.

Evaluation

A test harness for the harness – task suites, success criteria, regression detection.

Optimization

Prompt caching, tool caching, threading – make the harness affordable and fast.

10 modules build these end-to-end. **This is the talk's focus.**

Discipline 3 • Agentic engineering

Once you have an agent – a model wrapped in a harness – what do you do with it?

A • OUTWARD

Develop other products

Point the agent at the next codebase. Ship features, build infrastructure, author tooling.

Example: Peter Steinberg built openclaw by directing existing coding agents, then embedded a harness inside it.

B • RECURSIVE

Develop the agent itself

Point the agent at its own curriculum. Write a new module, refactor a component, raise the evals.

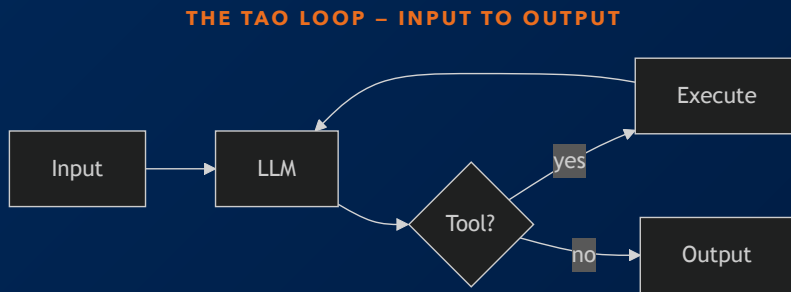
This repo and deck are built that way: Claude Code (a harness) running on Claude, driven by me.

Vibe coding's disciplined cousin.

Same fundamental move – have AI write the code – but with thought about what to ask, what tools to provide, how to verify, how to ship. **You're the human orchestrating agents to do the engineering.**

Module 1 • What is an agent?

concept only • modules/01-what-is-an-agent/



01 • REASONING

LLM call – the model

02 • THE BODY

Loop – the harness's control flow

03 • WORLD INTERFACE

Tools – the harness's hands

Three primitives. The harness is **two of them**.

Module 2 • An LLM call

harness component: model interface • modules/02-an-llm-call/ → llm_call_sync.py, llm_call_async.py

```
from anthropic import Anthropic
client = Anthropic(api_key=os.environ["ANTHROPIC_API_KEY"])

response = client.messages.create(
    model="claude-sonnet-4-5",
    max_tokens=1024,
    system="You are a helpful assistant.",
    messages=[{"role": "user", "content": "..."}],
)
print(response.content[0].text)
```

One HTTP POST. One JSON response. `content` is a list of blocks (text + optional tool requests).

Streaming version uses `messages.stream` + `await stream.get_final_message()` – text lands token-by-token, structured response captured at the end. **Every example downstream uses async streaming.**

Module 3 • Add a loop

harness component: control flow • modules/03-add-a-loop/ → stateless_chatbot.py

```
async def main():
    messages = []
    while True:
        user_input = input("> ")
        if user_input.lower() in ("/q", "exit"): break
        messages.append({"role": "user", "content": user_input})
        async with client.messages.stream(
            model="claude-sonnet-4-5",
            max_tokens=1024,
            system="You are a helpful assistant.",
            messages=messages,
        ) as stream:
            async for text in stream.text_stream:
                print(text, end="", flush=True)
            response = await stream.get_final_message()
            messages.append({"role": "assistant", "content": response.content[0].text})
```

The Messages API is stateless. The program holds the state. **Terminal is just our pick – the loop is environment-**

Module 4 • Add memory

harness component: memory + context management • modules/04-add-memory/ → stateful_chatbot.py

01 • SURVIVE A RESTART

Persistence

Save `messages.json` to disk.
Load at startup.

02 • FIT THE WINDOW

Token budget

Compute upfront. Walk past turns
newest-first until full.

03 • DON'T LOSE CONTEXT

Semantic recall

Summarize each turn, embed,
retrieve by similarity.

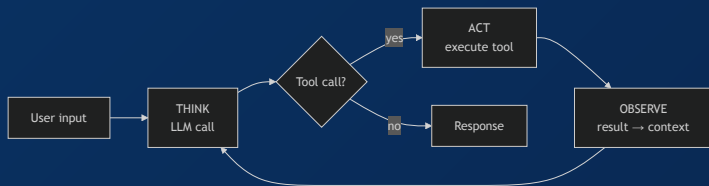
```
past_turn_budget = CONTEXT_BUDGET - MAX_RESPONSE_TOKENS  
                - tokens(system) - tokens(tools) - tokens(user_input)
```

`tiktoken cl100k_base` • `sentence-transformers all-MiniLM-L6-v2` • normalized vectors → dot product = cosine

Module 5 • Add tools • the agent moment

harness component: tool / action layer • modules/05-add-tools/ → agent.py

THE TAO LOOP



The model – not your code – decides what comes next.

The stateful chatbot becomes a stateful agent.

TOOLKIT • 6 TOOLS

read • grep • glob
write • edit • bash

HOW THEY SCALE

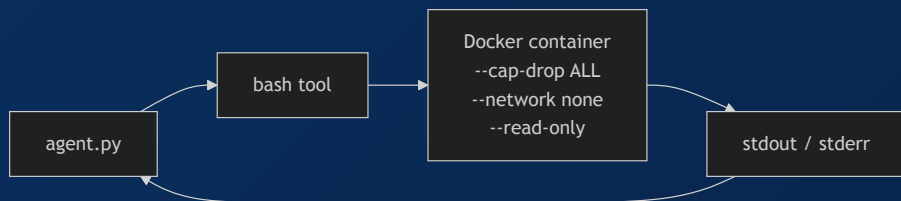
Registry collapses repeat plumbing.
Central executor catches all errors.
asyncio.gather dispatches in parallel.

Module 6 • Add sandboxing

harness component: execution environment • modules/06-add-sandboxing/ → sandbox_agent.py + Dockerfile.sandbox

The agent has a `bash` tool that runs **directly on the host**. The model can write your filesystem, install packages, exfiltrate data – by mistake or by prompt injection.

THE FIX • CONTAIN BASH IN DOCKER



Only `bash` is sandboxed. `read` / `write` / `edit` still touch the host – file editing has to be visible.

Module 7 • Add guardrails

harness component: `safety constraints · modules/07-add-guardrails/ → safe_agent.py`

01 · BEFORE THE ACTION

Approval gates

Before running a dangerous tool (`write` / `edit` / `bash`), prompt the human y/N.

02 · CAP THE RUNTIME

Loop bounds

`MAX_ITERATIONS` cap on the inner TAO loop. Stop before the agent burns budget.

03 · SURVIVE TRANSIENTS

Retry / backoff

Exponential backoff on transient API errors. Tool errors handled by the model.

Sandbox constrains **where** the agent can act. Guardrails constrain **whether** it gets to act at all.

Module 8 • Add observability

harness component: structured tracing • modules/08-add-observability/ → traced_agent.py

Every LLM call and tool call becomes a **structured span**. JSONL – one span per line.

```
{
  "name": "turn", "trace_id": "a1b2...", "duration_ms": 8240,
  "attributes": {"user_input": "what does foo.py import?", "iterations": 2},
  "children": [
    {"name": "memory.recall", "duration_ms": 0.02},
    {"name": "llm.call", "attributes": {"iteration": 0, "input_tokens": 1059},
      "children": [
        {"name": "tool.call", "attributes": {"tool.name": "read"}}
      ]
    },
    {"name": "llm.call", "attributes": {"iteration": 1, "input_tokens": 1183}},
    {"name": "guardrail.sentiment", "attributes": {"label": "POSITIVE"}},
    {"name": "guardrail.hallucination", "attributes": {"grounded": true}},
    {"name": "memory.summarize"}
  ]
}
```

Module 9 • Add evaluation

harness component: test infrastructure • modules/09-add-evaluation/ → evals/

A YAML CASE

```
id: find-imports
input: |
  list functions in foo.py
  that import requests
checks:
  - type: contains
    value: "fetch_user"
  - type: llm_judge
    rubric: |
      answer lists exactly the
      functions, no extras
```

THE RUNNER

- Subprocess per case (fresh state)
- N runs per case (default 3)
- Stochastic pass rate
- LLM-as-judge with Haiku
- Result file per timestamp
- `diff.py` flags >10% regression

```
uv run --project examples evals/run.py examples/production_agent.py
```

Module 10 • Add performance

harness component: production hardening • modules/10-add-performance/ → production_agent.py

01 • AMORTIZE INPUT COST

Prompt caching

Mark system + tool schemas `cache_control`.
Amortize input cost across many turns.

02 • DON'T PAY TWICE

Tool output caching

Two reads of the same file in one turn shouldn't pay twice. Content-addressed cache around `read` / `grep` / `glob`.

03 • OFF THE EVENT LOOP

Threading

CPU work (big regex trees, embedding inference) runs on a thread so concurrent tools aren't serialized behind it.

04 • ONE NAMED CALL SITE

Structured prompts • `assemble()`

One function brings together system, recalled memory, tool schemas, trimmed messages. One named call site.